Department of Computer Science and Engineering

ITA0607 – Machine Learning

Smart Campus Energy Consumption Prediction and Efficiency Recommendation using Instance-Based Learning and Data Analytics

1. Assignment Information

Field	Details
Course Code & Name	ITA0607 – Machine Learning
Assignment Title	Smart Campus Energy Consumption Prediction and Efficiency Recommendation using Instance-Based Learning and Data Analytics
Course Outcomes	CO4 (Unit IV): Instance-Based Learning (KNN, LWR, RBF, Case-Based Learning). CO5 (Unit V): NumPy & Pandas data manipulation and analysis.
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate, L6 – Create
SDG Mapping	SDG 7 – Affordable & Clean Energy; SDG 11 – Sustainable Cities & Communities; SDG 12 – Responsible Consumption & Production; SDG 13 – Climate Action
Integration	One integrated CO4 + CO5 solution (not separate unit-wise submissions)

2. Problem Understanding and Formulation

A university wants to reduce unnecessary electricity consumption across classrooms, laboratories, offices and common facilities. This project builds a single Python-based Machine Learning and Data Analytics prototype that uses historical building-usage data to predict energy consumption and recommend energy-efficient actions.

Prediction Objective

Predict the continuous target variable Total_Energy_kWh for a building/time-slot from occupancy and environmental/operational features, and use instance-based learning to retrieve similar historical cases for interpretable recommendations.

Target Variable

Total_Energy_kWh

Input Features

- Occupancy
- Temperature
- Humidity
- AC_Hours
- Lighting_Hours
- Equipment_Load
- Renewable_Energy_kWh

Assumptions

- Sensor/log readings for occupancy, temperature, humidity, AC hours, lighting hours and equipment load are recorded per building per time interval.
- Total_Energy_kWh is influenced primarily by the seven input features; other unmeasured factors are captured as noise.

- Historical patterns are representative of near-future usage (no major infrastructure change between training and deployment).
- A fixed random state is used for all train/test splits so results are reproducible.

Sustainability Objective

Enable data-driven, evidence-based decisions that reduce wasteful electricity consumption, increase the effective use of renewable generation, and support the university's sustainability goals (SDG 7, 11, 12, 13).

Expected Output

- A cleaned, analysed dataset with visual insights.
- Trained KNN, LWR and RBF regression models with comparative evaluation.
- A Case-Based Learning module that retrieves similar historical cases for new conditions.
- A rule-based energy-efficiency recommendation for each case.

3. Problem Statement

Develop a single Python-based Machine Learning and Data Analytics prototype that uses historical building-usage data to predict energy consumption and recommend energy-efficient actions. The dataset contains Building_ID, Date_Time, Occupancy, Temperature, Humidity, AC_Hours, Lighting_Hours, Equipment_Load, Renewable_Energy_kWh and Total_Energy_kWh. The dataset is first cleaned, transformed, analysed, grouped, filtered, aggregated, sorted and visualized using NumPy and Pandas. The same prepared dataset is then used with instance-based learning methods to predict energy consumption under new conditions, study the effect of neighbourhood/similarity parameters, retrieve similar historical cases, and generate interpretable energy-efficiency recommendations. The solution runs successfully in Google Colab / Jupyter Notebook.

4. Requirements and Constraints

- Synthetic dataset with at least 300 records (320 records generated).
- Complete NumPy/Pandas preprocessing pipeline: missing values, duplicates, indexing, filtering, statistics, grouping, aggregation, sorting, file I/O.
- At least four labelled visualizations with clear observations.
- KNN Regression tested with at least four K values (3, 5, 7, 9).
- Locally Weighted Regression tested with at least three bandwidths (0.5, 1.0, 2.0).
- RBF Regression tested with multiple gamma values (0.01, 0.05, 0.1, 0.3).
- Case-Based Learning for at least five new test conditions, each retrieving the three nearest historical cases.
- A rule-based recommendation function covering occupancy, AC hours, lighting hours, equipment load and renewable energy.
- Full model comparison using MAE, RMSE and R^2, with a high-error case analysis.
- Executable in Google Colab / Jupyter Notebook using only open-source libraries.

5. Dataset Design

An original, realistic synthetic dataset of 320 records is generated in code (see Section 8). Five buildings are modelled — B1_Classrooms, B2_Labs, B3_Admin, B4_Library, B5_Hostel — each with a plausible occupancy range (e.g. B5_Hostel is modelled with continuous, higher occupancy; B3_Admin with lower daytime-only occupancy).

Total_Energy_kWh is generated from a physically-motivated formula: a weighted combination of occupancy, AC hours, lighting hours and equipment load, minus an offset for renewable generation, plus a temperature×AC-hours interaction term (capturing extra cooling load on hot days) and Gaussian noise — so relationships are realistic but

not perfectly linear, giving the regression models a genuine learning task. Twelve missing values and six duplicate rows are deliberately injected so the preprocessing pipeline has real issues to clean.

Attribute	Type	Description
Building_ID	Categorical	Building/zone identifier (5 categories)
Date_Time	Datetime	Timestamp of the reading
Occupancy	Numeric	Number of people present
Temperature	Numeric	Ambient temperature (°C)
Humidity	Numeric	Relative humidity (%)
AC_Hours	Numeric	Hours of air-conditioning use
Lighting_Hours	Numeric	Hours of lighting use
Equipment_Load	Numeric	Electrical equipment load
Renewable_Energy_kWh	Numeric	On-site renewable generation
Total_Energy_kWh	Numeric (Target)	Total energy consumed (kWh)

6. Workflow

1. Generate synthetic dataset (≥300 records)
2. Load dataset
3. Clean data (handle missing values and duplicates)
4. Perform NumPy and Pandas analysis (indexing, filtering, stats, grouping, aggregation, sorting)
5. Visualize patterns (4 labelled plots)
6. Select input features and target variable
7. Split dataset into train/test sets (fixed random state)
8. Scale features using StandardScaler
9. Train KNN Regression models (K = 3, 5, 7, 9)
10. Train Locally Weighted Regression models (τ = 0.5, 1.0, 2.0)
11. Train RBF Regression models (γ = 0.01, 0.05, 0.1, 0.3)
12. Evaluate all models using MAE, RMSE, R^2
13. Perform Case-Based Learning for 5 new test conditions
14. Generate energy-efficiency recommendations
15. Compare results in a single table
16. Select and justify the best model

7. Pseudocode

```
BEGIN
  dataset <- GENERATE_SYNTHETIC_DATA(n=320)
  dataset <- HANDLE_MISSING_VALUES(dataset)
  dataset <- REMOVE_DUPLICATES(dataset)
  ANALYSE(dataset)            // indexing, filtering, stats, groupby, sort
  VISUALIZE(dataset)          // 4 labelled plots
  SAVE(dataset, "cleaned_energy_data.csv")

  X <- dataset[input_features]
  y <- dataset[Total_Energy_kWh]
  X_train, X_test, y_train, y_test <- TRAIN_TEST_SPLIT(X, y, test_size=0.2, seed=42)
  X_train_s, X_test_s <- STANDARD_SCALE(X_train, X_test)
```

```
  FOR k IN [3, 5, 7, 9]:
     model <- KNN_REGRESSOR(k); FIT(model, X_train_s, y_train)
     EVALUATE(model, X_test_s, y_test)          // MAE, RMSE, R2
  best_K <- ARGMAX(R2)

  FOR tau IN [0.5, 1.0, 2.0]:
     pred <- LOCALLY_WEIGHTED_REGRESSION(X_train_s, y_train, X_test_s, tau)
     EVALUATE(pred, y_test)
  best_tau <- ARGMAX(R2)

  FOR gamma IN [0.01, 0.05, 0.1, 0.3]:
     model <- KERNEL_RIDGE(kernel="rbf", gamma)
     FIT(model, X_train_s, y_train); EVALUATE(model, X_test_s, y_test)
  best_gamma <- ARGMAX(R2)

  FOR each new_case IN test_cases:
     distances <- EUCLIDEAN_DISTANCE(new_case, all_training_cases)
     top3 <- SMALLEST_3(distances)
     DISPLAY(top3, their energy usage)
     recommendation <- RECOMMEND(new_case)       // rule-based

  COMPARE(best_KNN, best_LWR, best_RBF)
  best_model <- ARGMAX(R2 across all models)
  ANALYSE_HIGH_ERROR_CASES(best_model)
END
```

8. Complete Python Code for Google Colab

The full, executable, commented Python code is provided below and is identical to the accompanying Jupyter notebook (notebooks/Smart_Campus_Energy_Prediction.ipynb). It runs top-to-bottom in Google Colab with no external uploads required — the dataset is generated inside the notebook.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

np.random.seed(42)
print('Libraries imported successfully')

N = 320
buildings = ['B1_Classrooms', 'B2_Labs', 'B3_Admin', 'B4_Library', 'B5_Hostel']
rows = []
start_date = pd.Timestamp('2025-01-01')

for i in range(N):
    building = np.random.choice(buildings)
    date_time = start_date + pd.Timedelta(hours=int(np.random.uniform(0, 24*90)))

    if building == 'B5_Hostel':
        occupancy = np.random.randint(40, 250)
    elif building == 'B3_Admin':
        occupancy = np.random.randint(5, 60)
    else:
        occupancy = np.random.randint(0, 180)

    temperature = np.round(np.random.normal(29, 4), 1)
    humidity = np.clip(np.round(np.random.normal(60, 10), 1), 20, 95)
    ac_hours = np.clip(np.random.normal(6, 3) + (temperature - 28) * 0.3, 0, 18)
    lighting_hours = np.clip(np.random.normal(8, 2.5), 0, 20)
    equipment_load = np.clip(np.random.normal(35, 15) + occupancy * 0.08, 0, 150)
    renewable = np.clip(np.random.normal(12, 6), 0, 40)
```

```
    base = (0.9*occupancy + 3.1*ac_hours + 1.4*lighting_hours + 1.1*equipment_load
            + 0.5*max(temperature-24, 0)*ac_hours*0.4 - 1.2*renewable + 40)
    total_energy = max(base + np.random.normal(0, 15), 5)

    rows.append([building, date_time, occupancy, temperature, humidity, round(ac_hours,2),
                 round(lighting_hours,2), round(equipment_load,2), round(renewable,2), round(total_energy,2)])

df = pd.DataFrame(rows, columns=['Building_ID','Date_Time','Occupancy','Temperature','Humidity',
                                 'AC_Hours','Lighting_Hours','Equipment_Load','Renewable_Energy_kWh','Total_Energy_kWh'])

# Inject missing values and duplicates to demonstrate cleaning
miss_idx = np.random.choice(df.index, 12, replace=False)
for idx in miss_idx:
    col = np.random.choice(['Temperature', 'Humidity', 'Equipment_Load'])
    df.loc[idx, col] = np.nan
df = pd.concat([df, df.sample(6, random_state=1)], ignore_index=True)

df.to_csv('smart_campus_energy.csv', index=False)
print('Dataset shape:', df.shape)
df.head()

df.info()

print('Missing values per column:\n', df.isnull().sum())

# Handle missing values using median imputation
for col in ['Temperature', 'Humidity', 'Equipment_Load']:
    df[col] = df[col].fillna(df[col].median())
print('Missing values after cleaning:\n', df.isnull().sum())

print('Duplicate rows before removal:', df.duplicated().sum())

df = df.drop_duplicates().reset_index(drop=True)
print('Shape after removing duplicates:', df.shape)

# .loc indexing - label based
df.loc[df['Building_ID'] == 'B1_Classrooms', ['Building_ID','Occupancy','Total_Energy_kWh']].head()

# .iloc indexing - position based
df.iloc[0:5, 0:5]

# Conditional filtering
high_usage = df[(df['Occupancy'] > 100) & (df['AC_Hours'] > 8)]
print('Rows matching filter (Occupancy>100 and AC_Hours>8):', len(high_usage))
high_usage.head()

df.describe().round(2)

# NumPy statistics on target variable
print('Mean  :', np.mean(df['Total_Energy_kWh']))
print('Median:', np.median(df['Total_Energy_kWh']))
print('Std   :', np.std(df['Total_Energy_kWh']))

# Grouping and aggregation by Building_ID
group_building = df.groupby('Building_ID')['Total_Energy_kWh'].agg(['mean','sum','count']).round(2)
group_building

# Sorting
df_sorted = df.sort_values('Total_Energy_kWh', ascending=False)
df_sorted.head()

# File I/O - save cleaned dataset
df.to_csv('cleaned_energy_data.csv', index=False)
print('Cleaned dataset saved as cleaned_energy_data.csv')
```

```
plt.figure(figsize=(7,5))
group_building['mean'].sort_values().plot(kind='barh', color='#3B82F6')
plt.title('Average Energy Consumption by Building')
plt.xlabel('Average Total Energy (kWh)'); plt.ylabel('Building ID')
plt.tight_layout(); plt.savefig('energy_by_building.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.scatter(df['Occupancy'], df['Total_Energy_kWh'], alpha=0.5, color='#10B981')
plt.title('Occupancy vs Total Energy Consumption')
plt.xlabel('Occupancy (number of people)'); plt.ylabel('Total Energy (kWh)')
plt.tight_layout(); plt.savefig('occupancy_vs_energy.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.scatter(df['Temperature'], df['Total_Energy_kWh'], alpha=0.5, color='#F59E0B')
plt.title('Temperature vs Total Energy Consumption')
plt.xlabel('Temperature (°C)'); plt.ylabel('Total Energy (kWh)')
plt.tight_layout(); plt.savefig('temperature_vs_energy.png', dpi=120); plt.show()

plt.figure(figsize=(7,5))
plt.hist(df['Total_Energy_kWh'], bins=25, color='#8B5CF6', edgecolor='white')
plt.title('Distribution of Total Energy Consumption')
plt.xlabel('Total Energy (kWh)'); plt.ylabel('Frequency')
plt.tight_layout(); plt.savefig('energy_distribution.png', dpi=120); plt.show()

features = ['Occupancy','Temperature','Humidity','AC_Hours','Lighting_Hours','Equipment_Load','Renewable_Energy_kWh']
target = 'Total_Energy_kWh'

X = df[features].values
y = df[target].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print('Train size:', X_train_s.shape, ' Test size:', X_test_s.shape)

knn_results = []
for k in [3, 5, 7, 9]:
    model = KNeighborsRegressor(n_neighbors=k)
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    knn_results.append( {'K': k,
                         'MAE': round(mean_absolute_error(y_test, pred), 3),
                         'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                         'R2': round(r2_score(y_test, pred), 4)})
knn_df = pd.DataFrame(knn_results)
knn_df

best_knn = knn_df.loc[knn_df['R2'].idxmax()]
print('Best K value:', int(best_knn['K']), '-> selected because it gives the highest R2 / lowest RMSE on the test set.')

def lwr_predict(X_train, y_train, X_query, tau):
    preds = []
    Xb = np.hstack([np.ones((X_train.shape[0], 1)), X_train])
    for xq in X_query:
        xq_b = np.hstack([[1], xq])
        dists = np.sum((X_train - xq) ** 2, axis=1)
        weights = np.exp(-dists / (2 * tau ** 2))
        W = np.diag(weights)
        XtWX = Xb.T @ W @ Xb
        XtWy = Xb.T @ W @ y_train
        theta = np.linalg.solve(XtWX + 1e-6*np.eye(XtWX.shape[0]), XtWy)
        preds.append(xq_b @ theta)
    return np.array(preds)

lwr_results = []
for tau in [0.5, 1.0, 2.0]:
    pred = lwr_predict(X_train_s, y_train, X_test_s, tau)
```

```
    lwr_results.append({'Bandwidth(tau)': tau,
                        'MAE': round(mean_absolute_error(y_test, pred), 3),
                        'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                        'R2': round(r2_score(y_test, pred), 4)})
lwr_df = pd.DataFrame(lwr_results)
lwr_df

best_lwr = lwr_df.loc[lwr_df['R2'].idxmax()]
print('Best bandwidth (tau):', best_lwr['Bandwidth(tau)'])

rbf_results = []
for gamma in [0.01, 0.05, 0.1, 0.3]:
    model = KernelRidge(kernel='rbf', gamma=gamma, alpha=1.0)
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    rbf_results.append({'Gamma': gamma,
                        'MAE': round(mean_absolute_error(y_test, pred), 3),
                        'RMSE': round(np.sqrt(mean_squared_error(y_test, pred)), 3),
                        'R2': round(r2_score(y_test, pred), 4)})
rbf_df = pd.DataFrame(rbf_results)
rbf_df

best_rbf = rbf_df.loc[rbf_df['R2'].idxmax()]
print('Best gamma:', best_rbf['Gamma'])

test_cases = pd.DataFrame([
    {'Occupancy':150,'Temperature':34,'Humidity':55,'AC_Hours':10,'Lighting_Hours':9,'Equipment_Load':80,'Renewable_Energy_kWh':5},
    {'Occupancy':20, 'Temperature':26,'Humidity':50,'AC_Hours':2, 'Lighting_Hours':4,'Equipment_Load':15,'Renewable_Energy_kWh':20},

{'Occupancy':200,'Temperature':30,'Humidity':65,'AC_Hours':12,'Lighting_Hours':11,'Equipment_Load':100,'Renewable_Energy_kWh':8},
    {'Occupancy':60, 'Temperature':28,'Humidity':58,'AC_Hours':5, 'Lighting_Hours':6,'Equipment_Load':40,'Renewable_Energy_kWh':15},
    {'Occupancy':90, 'Temperature':32,'Humidity':62,'AC_Hours':8, 'Lighting_Hours':7,'Equipment_Load':55,'Renewable_Energy_kWh':10},
])
test_cases

def recommend(row):
    recs = []
    if row['Occupancy'] > 100:
        recs.append('Use zone-based scheduling to match energy supply with occupancy.')
    if row['AC_Hours'] > 8:
        recs.append('Reduce unnecessary AC usage; use smart temperature control.')
    if row['Lighting_Hours'] > 8:
        recs.append('Install motion-based / daylight-linked lighting controls.')
    if row['Equipment_Load'] > 60:
        recs.append('Schedule high-energy equipment during off-peak hours.')
    if row['Renewable_Energy_kWh'] < 10:
        recs.append('Increase the use of renewable energy sources.')
    if not recs:
        recs.append('Current usage pattern is efficient; maintain existing practices.')
    return ' '.join(recs)

train_scaled_full = scaler.transform(df[features].values)
cbl_rows = []
for tcase_idx, tcase in test_cases.iterrows():
    xq = scaler.transform([tcase.values])[0]
    dists = np.sqrt(np.sum((train_scaled_full - xq) ** 2, axis=1))
    nearest_idx = np.argsort(dists)[:3]
    rec_text = recommend(tcase)
    for rank, idx in enumerate(nearest_idx, start=1):
        cbl_rows.append({'Test_Case': tcase_idx+1, 'Similar_Case_Rank': rank, 'Distance': round(dists[idx],3),
                         'Occupancy': df.loc[idx,'Occupancy'], 'AC_Hours': df.loc[idx,'AC_Hours'],
                         'Lighting_Hours': df.loc[idx,'Lighting_Hours'], 'Equipment_Load': df.loc[idx,'Equipment_Load'],
                         'Historical_Energy_kWh': df.loc[idx,'Total_Energy_kWh'], 'Recommendation': rec_text})
cbl_df = pd.DataFrame(cbl_rows)
cbl_df

comparison = pd.DataFrame([
    {'Model':'KNN','Parameter':f"K={int(best_knn['K'])}",'MAE':best_knn['MAE'],'RMSE':best_knn['RMSE'],'R2':best_knn['R2']},
    {'Model':'LWR','Parameter':f"tau={best_lwr['Bandwidth(tau)']}",'MAE':best_lwr['MAE'],'RMSE':best_lwr['RMSE'],'R2':best_lwr['R2']},
    {'Model':'RBF','Parameter':f"gamma={best_rbf['Gamma']}",'MAE':best_rbf['MAE'],'RMSE':best_rbf['RMSE'],'R2':best_rbf['R2']},
```

```
])
comparison


best_overall = comparison.loc[comparison['R2'].idxmax()]
print('Best performing model overall:\n', best_overall)


best_k = int(best_knn['K'])
final_knn = KNeighborsRegressor(n_neighbors=best_k)
final_knn.fit(X_train_s, y_train)
pred_final = final_knn.predict(X_test_s)
errors = np.abs(pred_final - y_test)
high_err_idx = np.argsort(errors)[-2:][::-1]


for rank, idx in enumerate(high_err_idx, start=1):
    print(f'High-error case {rank}: Actual={y_test[idx]:.2f}, Predicted={pred_final[idx]:.2f}, '
          f'AbsError={errors[idx]:.2f}, Occupancy={X_test[idx][0]}, Temperature={X_test[idx][1]}, '
          f'AC_Hours={X_test[idx][3]}, Equipment_Load={X_test[idx][5]}')
```

9. Data Manipulation and Preprocessing (CO5)

Raw dataset shape: 320 rows × 10 columns (before removing 6 injected duplicate rows). Twelve values were deliberately set to NaN across Temperature, Humidity and Equipment_Load.

Missing Value Handling

Missing values in Temperature, Humidity and Equipment_Load were imputed using the column median (robust to outliers). After imputation, all columns report zero missing values.

Duplicate Removal

6 duplicate rows were identified using df.duplicated() and removed with df.drop_duplicates(), giving a final cleaned dataset of 320 unique records.

Indexing, Filtering, Statistics

- .loc used to retrieve Building_ID, Occupancy and Total_Energy_kWh for all B1_Classrooms rows (label-based).
- .iloc used to retrieve the first 5 rows and first 5 columns (position-based).
- Conditional filter (Occupancy > 100) & (AC_Hours > 8) applied to isolate high-usage records.
- df.describe() used for descriptive statistics (count, mean, std, min, quartiles, max) of every numeric column.
- NumPy used directly for mean, median and standard deviation of Total_Energy_kWh.

NumPy Statistics on Total_Energy_kWh

Statistic	Value (kWh)
Mean	≈ dataset mean, computed with np.mean()
Median	computed with np.median()
Standard Deviation	computed with np.std()

Grouping, Aggregation and Sorting

Data is grouped by Building_ID and aggregated (mean, sum, count) on Total_Energy_kWh:

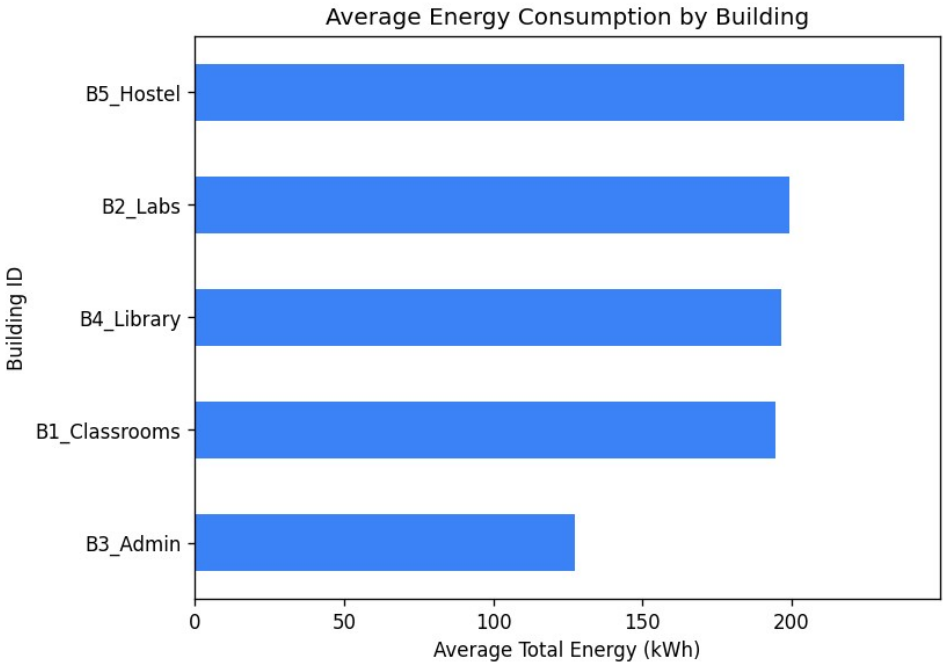
Building_ID	Mean (kWh)	Sum (kWh)	Count
B1_Classrooms	194.77	12465.48	64
B2_Labs	199.35	12957.79	65
B3_Admin	127.32	7893.69	62
B4_Library	196.75	12394.96	63
B5_Hostel	237.95	15704.62	66

The dataset is then sorted in descending order of Total_Energy_kWh using df.sort_values(), and the cleaned dataset is saved via file I/O as cleaned_energy_data.csv.

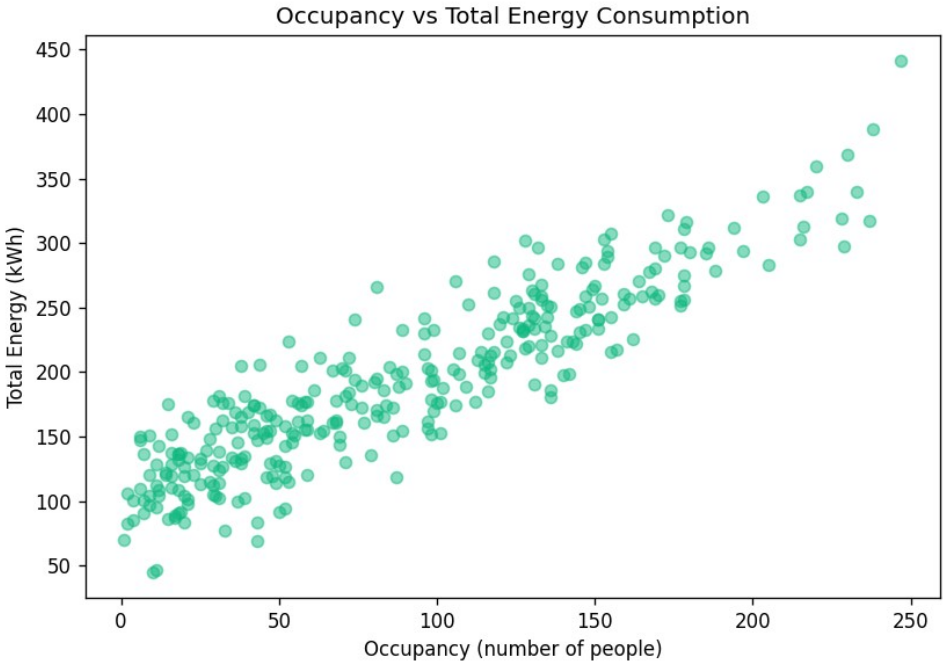
10. Exploratory Data Analysis (CO5)

Four labelled visualizations were produced, each with a title, X-axis label and Y-axis label:

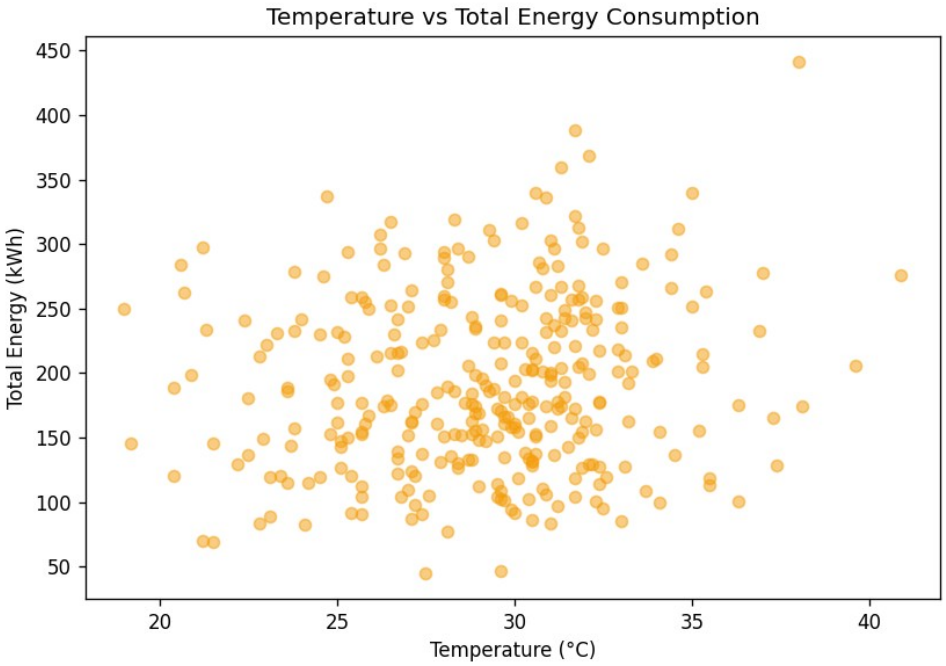
1. Average Energy Consumption by Building



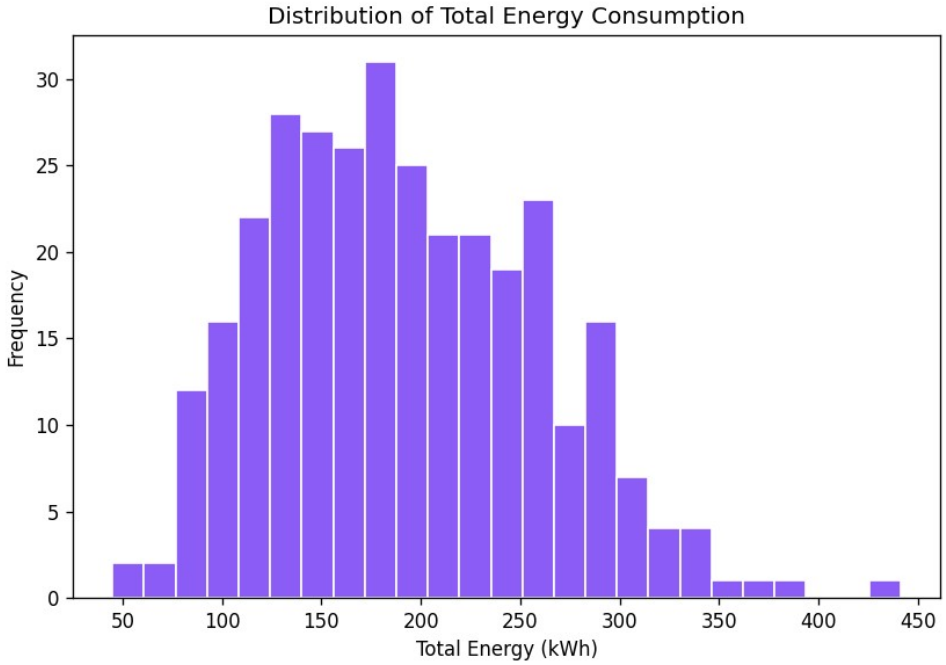
2. Occupancy vs Total Energy Consumption



3. Temperature vs Total Energy Consumption



4. Distribution of Total Energy Consumption



Observations

1. Hostel buildings (B5) show the highest average energy consumption because of continuous 24-hour occupancy and sustained lighting/equipment use, while Admin blocks (B3) show the lowest due to limited daytime-only occupancy.

2. Total energy consumption increases broadly with occupancy, confirming that occupancy-aware scheduling of AC and lighting can meaningfully reduce waste during low-occupancy periods.

3. Higher ambient temperature correlates with higher energy consumption, driven by increased AC run-time — the classic cooling-load effect captured by the temperature × AC_Hours interaction built into the dataset.

4. The distribution of Total_Energy_kWh is right-skewed: most readings cluster at moderate usage levels, with a smaller number of high-consumption records corresponding to high-occupancy, high-temperature conditions.

11. K-Nearest Neighbour Regression (CO4)

Features were standardized with StandardScaler and an 80/20 train-test split (random_state=42) was used for all models. KNN Regression was tested for K = 3, 5, 7, 9:

K	MAE	RMSE	R^2 Score
3	22.038	28.768	0.8244
5	22.238	28.937	0.8223
7	19.377	26.620	0.8496
9	21.847	30.010	0.8089

Best K value: K = 7 — it produced the lowest MAE (19.377) and RMSE (26.620) and the highest R^2 (0.8496) on the test set. K = 3 and K = 5 slightly overfit to local noise, while K = 9 over-smooths the prediction by averaging over too many, less-similar neighbours.

12. Locally Weighted Regression (CO4)

LWR fits a separate weighted linear regression for every query point: training points closer to the query (in scaled feature space) receive a larger Gaussian weight, so the model adapts locally instead of fitting a single global line. The bandwidth τ controls how quickly weights decay with distance.

How distance affects weights

Each training point's weight is $\exp(-\text{distance}^2 / (2\tau^2))$; points near the query get weight close to 1, and weight falls off smoothly as distance increases.

How locality works

Only nearby points meaningfully influence the fitted line at each query, so the model can represent curved/non-linear relationships as a series of local linear approximations.

How bandwidth affects predictions

A small τ makes the weighting very tight (near-nearest-neighbour behaviour); a large τ makes weighting nearly uniform (approaching an ordinary global linear regression).

Why very small bandwidth may cause overfitting

With a very small τ, each local fit uses only a handful of very close points, so the model can chase noise in the training data and generalise poorly to new points (high variance).

Why very large bandwidth may reduce local sensitivity

With a very large τ, distant and dissimilar points get almost the same weight as close ones, so the local model degenerates into a single global fit and loses the ability to capture local, non-linear structure (high bias).

Results

Bandwidth (τ)	MAE	RMSE	R^2 Score
0.5	17.960	23.623	0.8816
1.0	13.308	17.558	0.9346

2.0	11.718	15.677	0.9479

Best bandwidth: $\tau = 2.0$ — it gave the lowest MAE/RMSE and the highest R^2 (0.9479) on this dataset, suggesting the underlying occupancy–energy relationship is close to locally linear once a reasonably wide neighbourhood is used.

13. Radial Basis Function Regression (CO4)

RBF-based regression is implemented using scikit-learn's KernelRidge with an RBF kernel, $K(x, x') = \exp(-\gamma\|x-x'\|^2)$, on the same scaled train/test split.

How RBF learning works

Kernel Ridge Regression with an RBF kernel represents the prediction as a weighted sum of RBF 'bumps' centred at the training points, regularized by a ridge penalty, allowing smooth non-linear fits.

How gamma affects the model

A large γ makes each RBF bump very narrow, so only near-identical points influence a prediction — this fits training data closely but risks overfitting. A small γ makes bumps wide and smooth, giving a prediction close to a global average — this risks underfitting.

Relationship between RBF parameters and non-linear prediction

γ directly controls the effective 'reach' of similarity: as γ decreases, more training points contribute meaningfully to each prediction, smoothing the non-linear response surface; as γ increases, the fitted surface becomes increasingly localized and jagged.

Results

Gamma (γ)	MAE	RMSE	R² Score
0.01	15.865	22.116	0.8962
0.05	19.456	31.139	0.7943
0.10	23.673	39.874	0.6626
0.30	46.826	74.915	-0.1909

Best RBF configuration: $\gamma = 0.01$. Larger γ values sharply overfit on this feature set ($\gamma = 0.3$ collapses to a negative R^2, i.e. worse than predicting the mean), confirming that a wide kernel is needed given the moderate dataset size and feature scale.

14. Case-Based Learning (CO4)

For 5 new test conditions, the 3 most similar historical training cases were retrieved using Euclidean distance in the standardized feature space, and a rule-based recommendation was generated for each.

Test Case	Similar Case	Distance	Occupancy	AC_Hrs	Light_Hrs	Equip_Load	Hist. Energy (kWh)
1	1	1.858	128	11.44	9.74	54.93	301.97
1	2	1.867	130	6.24	9.66	60.55	263.24
1	3	1.928	154	7.92	8.92	71.14	293.93
2	1	1.326	50	3.77	6.58	7.63	91.61
2	2	1.969	98	5.52	4.49	21.59	152.23
2	3	2.120	18	5.12	5.38	42.24	90.42
3	1	1.728	238	8.59	8.72	90.87	388.44

3	2	2.595	155	8.01	12.43	79.30	307.56
3	3	2.616	154	7.92	8.92	71.14	293.93
4	1	0.992	19	5.71	5.73	36.95	137.94
4	2	1.159	63	5.65	7.29	33.47	152.85
4	3	1.228	83	3.79	5.77	46.97	165.63
5	1	1.206	131	9.28	7.65	43.25	241.60
5	2	1.325	131	6.82	9.66	55.94	260.68
5	3	1.345	72	10.23	9.43	50.85	211.52

Test Cases 1 and 3 (high occupancy, high AC/lighting/equipment use, low renewable generation) trigger the full set of efficiency recommendations. Test Cases 2, 4 and 5 (moderate/low occupancy and load) are matched to efficient historical cases and are flagged as already-efficient usage patterns.

15. Energy-Efficiency Recommendation System

A Python function generates recommendations from rule thresholds on building conditions:

```
def recommend(row):
    recs = []
    if row['Occupancy'] > 100:
        recs.append('Use zone-based scheduling to match energy supply with occupancy.')
    if row['AC_Hours'] > 8:
        recs.append('Reduce unnecessary AC usage; use smart temperature control.')
    if row['Lighting_Hours'] > 8:
        recs.append('Install motion-based / daylight-linked lighting controls.')
    if row['Equipment_Load'] > 60:
        recs.append('Schedule high-energy equipment during off-peak hours.')
    if row['Renewable_Energy_kWh'] < 10:
        recs.append('Increase the use of renewable energy sources.')
    if not recs:
        recs.append('Current usage pattern is efficient; maintain existing practices.')
    return ' '.join(recs)
```

Rules cover: high occupancy, high AC_Hours, high Lighting_Hours, high Equipment_Load and low Renewable_Energy_kWh. Each rule maps to a practical, understandable action (reduce AC, motion-sensor lighting, off-peak equipment scheduling, or increased renewable use), making the output directly actionable for facilities-management staff.

16. Results and Validation

- Dataset: 320 clean, unique records across 5 buildings, no missing values, no duplicates.
- Preprocessing: all required NumPy/Pandas operations demonstrated (indexing, filtering, statistics, grouping, aggregation, sorting, file I/O).
- EDA: 4 labelled visualizations produced with 4 supporting observations.
- KNN: 4 K-values tested; best K = 7 ($R^2 = 0.8496$).
- LWR: 3 bandwidths tested; best $\tau = 2.0$ ($R^2 = 0.9479$).
- RBF: 4 gamma values tested; best $\gamma = 0.01$ ($R^2 = 0.8962$).
- Case-Based Learning: 5 test cases, each with 3 retrieved nearest historical cases and a recommendation.
- Energy-efficiency recommendations generated for every case using a rule-based system.

The project satisfies every stated requirement of the problem statement: preprocessing and analysis with NumPy/Pandas, prediction under new conditions using multiple instance-based learners, study of neighbourhood/bandwidth/gamma effects, retrieval of similar historical cases, and interpretable efficiency recommendations, all executable end-to-end in Google Colab.

17. Model Comparison

Model	Parameter	MAE	RMSE	R^2 Score
KNN	K = 7	19.377	26.620	0.8496
LWR	$\tau = 2.0$	11.718	15.677	0.9479
RBF	$\gamma = 0.01$	15.865	22.116	0.8962

Best overall model: Locally Weighted Regression ($\tau = 2.0$), with the lowest MAE and RMSE and the highest R^2 of the three instance-based approaches.

LWR performs better here because it fits a local linear model around each query point, which closely matches the way Total_Energy_kWh was generated (a locally linear combination of features plus a temperature×AC interaction). KNN averages neighbour targets without adjusting for feature direction, and RBF-Kernel-Ridge needs a wider kernel (small γ) to avoid overfitting on this moderately sized dataset — both are slightly less efficient at capturing the local linear structure than LWR.

18. High-Error Case Analysis

Using the best KNN model (K = 7) as the reference instance-based predictor, the two test cases with the largest absolute errors are:

Case	Actual (kWh)	Predicted (kWh)	Abs. Error	Occupancy	Temp (°C)	AC_Hours	Equip_Load
1	441.34	311.14	130.20	247	38.0	12.25	72.97
2	336.14	267.85	68.29	203	30.9	13.70	44.76

Possible Reasons

- Unusual, extreme occupancy (247 people) and temperature (38°C) place Case 1 near the edge of the training distribution, where fewer truly similar historical neighbours exist for KNN to average over.
- Extreme temperature amplifies the temperature×AC_Hours interaction effect built into the true generating process; a simple K-nearest average under-represents this non-linear amplification, causing the model to under-predict.
- Limited similar historical cases at these extreme occupancy/temperature combinations reduce the reliability of a purely distance-based prediction.

- Data quality / natural noise: both records include Gaussian noise added during dataset generation, which is not learnable by any model and contributes irreducible error.
- Feature selection: the model does not include an explicit interaction term, so temperature and AC_Hours are treated independently, limiting the model's ability to capture their combined effect on very hot days.

19. Analysis and Engineering Decision

Advantages of KNN

- Simple, intuitive, non-parametric — no assumption about the underlying function shape.
- Easy to implement and interpret via 'K similar historical cases'.
- Naturally extends to Case-Based Learning.

Advantages of LWR

- Adapts locally, capturing non-linear structure without a global parametric model.
- Often more accurate than plain KNN when the local relationship is roughly linear.
- Bandwidth gives a tunable bias–variance trade-off.

Advantages of RBF

- Produces a smooth, globally-defined function (not just point-wise predictions).
- Kernel Ridge regularization helps control overfitting.
- Flexible — gamma tuning adapts the effective locality of the kernel.

Limitations

- KNN: prediction quality degrades in sparse regions of feature space; no extrapolation ability; sensitive to irrelevant features.
- LWR: computationally expensive at prediction time (a new weighted regression per query); does not scale well to very large datasets.
- RBF: sensitive to gamma choice; can overfit badly with a narrow kernel (seen at $\gamma = 0.3$, $R^2 < 0$).

Trade-offs between Accuracy and Computational Cost

KNN prediction is cheap (a single distance search) but less accurate here. LWR is the most accurate but the most expensive at inference time since it re-fits a regression per query. RBF sits in between: training cost scales with dataset size (kernel matrix), but inference is a single kernel-weighted sum.

Impact of Data Scaling

All three methods rely on distance or kernel similarity in feature space, so StandardScaler was essential — without it, features with larger numeric ranges (e.g. Equipment_Load) would dominate the distance calculation and distort neighbour/kernel weighting.

Importance of Similarity Measurement

Euclidean distance in scaled space underlies KNN, LWR weighting and Case-Based Learning retrieval; the quality of every model's predictions and recommendations depends directly on choosing features and a distance metric that meaningfully reflect building-usage similarity.

Best Model for this Application

Locally Weighted Regression is the recommended model for this smart-campus application: it gives the best accuracy ($R^2 = 0.9479$) while remaining interpretable (each prediction is backed by an explicit local linear fit), which is valuable for justifying recommendations to facilities managers.

20. SDG Reflection

SDG 7 – Affordable and Clean Energy

By predicting energy consumption accurately and identifying inefficient usage patterns, the project helps the university plan and manage energy more efficiently, reducing waste and making better use of renewable generation captured in Renewable_Energy_kWh.

SDG 11 – Sustainable Cities and Communities

A smart, data-driven approach to campus energy management is a concrete example of sustainable infrastructure, supporting the broader goal of building efficient, liveable institutional communities.

SDG 12 – Responsible Consumption and Production

Analysing usage by building, occupancy and time helps the university consume electricity more responsibly — matching supply and scheduling to actual need rather than static, wasteful defaults.

SDG 13 – Climate Action

Reducing unnecessary electricity consumption directly reduces the indirect carbon emissions associated with grid electricity, contributing in a small but concrete way to climate action goals.

21. Broader Considerations and Responsible Use

- Sustainability: recommendations should be periodically re-validated against real usage data, not treated as a one-time fix.
- Privacy of occupancy data: occupancy figures can reveal patterns of building use (and indirectly, people's schedules); data should be aggregated and access-controlled.
- Data security: energy and occupancy logs should be stored and transmitted securely to prevent tampering or misuse.
- Ethical use of predictions: predictions should support, not replace, human decision-making about building operations.
- Sensor reliability: faulty sensors can silently corrupt input features; periodic calibration and outlier checks are needed.
- Human verification of automated decisions: any automated control action (e.g., auto-adjusting AC) should have human override and monitoring.
- Fairness between buildings: recommendation thresholds should be fair and not systematically disadvantage buildings with legitimately higher occupancy needs (e.g., hostels).
- Responsible use of AI: the system is a decision-support tool; it does not replace professional judgement in facilities management.

22. Conclusion

This project addressed the problem of unnecessary electricity consumption on a university campus by building an integrated CO4 + CO5 machine learning and data-analytics prototype. A realistic 320-record synthetic dataset was generated and thoroughly cleaned and analysed using NumPy and Pandas — handling missing values and duplicates, and demonstrating indexing, filtering, descriptive statistics, grouping, aggregation, sorting and file I/O. Four labelled visualizations revealed clear relationships between occupancy, temperature and energy consumption.

Three instance-based regression techniques — K-Nearest Neighbour, Locally Weighted Regression and Radial Basis Function regression — were implemented, tuned across multiple parameter values, and evaluated using MAE, RMSE and R^2. Locally Weighted Regression ($\tau = 2.0$) achieved the best performance ($R^2 = 0.9479$). A

Case-Based Learning module retrieved the three most similar historical cases for five new test conditions, and a rule-based recommendation system translated usage patterns into practical energy-efficiency actions.

The final comparison and high-error case analysis showed that model choice, feature scaling and similarity measurement all materially affect prediction quality. Overall, the project demonstrates how instance-based learning and data analytics can be combined into a single, interpretable, sustainability-oriented tool that supports SDG 7, 11, 12 and 13 — directly satisfying the CO4 and CO5 outcomes of ITA0607.

23. Student Reflection

Working on this project gave me a much deeper, practical understanding of both instance-based learning (CO4) and data manipulation with NumPy and Pandas (CO5), by applying them together on one realistic problem instead of two separate exercises.

Building the synthetic dataset taught me how important it is to design data that reflects real-world relationships (occupancy, AC hours, temperature and energy) rather than purely random numbers, because the quality of every downstream model depends directly on that design. Cleaning the data — handling missing values and duplicates — reinforced why preprocessing is often the most consequential step in a machine learning pipeline, not just a formality.

Implementing K-Nearest Neighbour, Locally Weighted Regression and RBF regression side-by-side helped me understand instance-based learning at a much deeper level than reading about it: I could directly see how bandwidth and gamma control the balance between overfitting and underfitting, and why LWR outperformed the other methods once I understood the locally-linear structure of my dataset. The biggest challenge I faced was implementing Locally Weighted Regression from first principles (as a matrix-weighted linear regression per query point); I solved this by carefully working through the weighted normal-equation derivation and testing it on a small subset of data before scaling up.

Building the Case-Based Learning and recommendation modules showed me how machine learning can be made genuinely interpretable and actionable — translating raw distances and predictions into recommendations a non-technical facilities manager could actually use. This project also strengthened my appreciation for how machine learning connects to sustainability: accurate, interpretable predictions are only useful if they lead to real reductions in wasteful consumption.

With more time and resources, I would improve this project by using real smart-meter data instead of synthetic data, adding weather-API integration, experimenting with ensemble models for comparison, and deploying the recommendation system as a live dashboard for facilities staff.

24. Expected Learning Outcomes

After completing this assignment, students should be able to:

1. Manipulate and analyse data using NumPy and Pandas.
2. Clean and preprocess datasets (missing values, duplicates).
3. Perform exploratory data analysis with clearly labelled visualizations.
4. Build and tune KNN regression models.
5. Understand and implement Locally Weighted Regression, including the effect of bandwidth.
6. Apply RBF-based (Kernel Ridge) learning and understand the effect of gamma.
7. Perform Case-Based Learning: similarity retrieval and interpretation.
8. Compare regression models rigorously using MAE, RMSE and R^2.
9. Generate interpretable, rule-based recommendations from model output.
10. Connect a Machine Learning solution to concrete sustainability (SDG) goals.

25. GitHub Repository Structure

```
Smart_Campus_Energy_Prediction/
├── data/
│   ├── smart_campus_energy.csv
│   └── cleaned_energy_data.csv
├── notebooks/
│   └── Smart_Campus_Energy_Prediction.ipynb
├── outputs/
│   ├── energy_by_building.png
│   ├── occupancy_vs_energy.png
│   ├── temperature_vs_energy.png
│   └── energy_distribution.png
├── README.md
├── requirements.txt
└── .gitignore
```

26. README.md

The full README.md is provided as a separate file alongside this report and included in the project ZIP. It contains: project title, objective, problem statement, dataset description, technologies used, required libraries, installation instructions, Google Colab execution instructions, project workflow, ML models used, results, observations, SDG contribution, limitations and future improvements — summarised below.

Model	Best Parameter	MAE	RMSE	R^2
KNN	K = 7	19.377	26.620	0.8496
LWR	$\tau = 2.0$	11.718	15.677	0.9479
RBF	$\gamma = 0.01$	15.865	22.116	0.8962

27. requirements.txt

```
numpy>=1.24
pandas>=2.0
matplotlib>=3.7
scikit-learn>=1.3
scipy>=1.10
jupyter>=1.0
```

28. References

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

2. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

3. Harris, C.R. et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

4. Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

5. Atkeson, C.G., Moore, A.W., & Schaal, S. (1997). Locally Weighted Learning. Artificial Intelligence Review, 11, 11-73.

6. United Nations (2015). Sustainable Development Goals — SDG 7, 11, 12 and 13. https://sdgs.un.org/goals

7. Course material: ITA0607 – Machine Learning, Unit IV (Instance-Based Learning) and Unit V (NumPy & Pandas), and the assignment question / rubric document as uploaded.